Lehrerexemplar · Nachschreibklausur · Welt 6 – Der Endgegner · DS 39–40

Erwartungshorizont: Die Kletterhalle

Erwartungshorizont / Bewertungsbogen für _____

I, II, III: erreichbare Bewertungseinheiten im jeweiligen Anforderungsbereich · BE: vom Prüfling erreichte Bewertungseinheiten

Erwartete Prüfungsleistungen (Kernpunkte)	I	II	III	BE
a) Konstruktoren und Sichtbarkeit erläutern • Konstruktor erzeugt ein Objekt und initialisiert die Attribute • c Mitglied(name): Name gesetzt, 0 Punkte, leeres Tourenbuch • c Mitglied(name, punkte): zusätzlich Startpunktzahl; Überladen, Auswahl über die Parameterliste, sinnvoller Anwendungsfall • – privat (nur innerhalb der Klasse), + öffentlich; Kapselung (Zugriff nur über Operationen)	3	1		
b) Klassendiagramm um die Klasse Boulder ergänzen • Klassenkarte Boulder mit – zonen: Ganzzahl, Konstruktor mit allen vier Parametern, + getZonen(): Ganzzahl • Vererbungspfeil mit hohler Dreiecksspitze von Boulder zu Route • Begründung: „ist ein“, Wiederverwendung der geerbten Attribute/Operationen, Speicherung in routen möglich	1	2		
c) Implementierung: Stapel auswerten und unverändert lassen • Methodenkopf int hoechsterGrad(int k); Rückgabe in jedem Pfad • Hilfsstapel; Entnehmen mit pop und Cast (Route) • Schleife endet nach k Routen oder bei leerem Tourenbuch; Leerabfrage vor jedem pop • Maximum über getGrad() mit geeignetem Startwert 0 • vollständiges Zurückschichten aus dem Hilfsstapel – Tourenbuch unverändert • Rückgabe des Maximums, 0 bei leerem Tourenbuch bzw. k ≤ 0 • Randfälle (k größer als die Anzahl, k = 0, leer) ohne Laufzeitfehler; keine Rückgabe vor dem Zurückschichten	1	5	1	
d) Struktogramm: aus einer Schlange entfernen, Reihenfolge erhalten • Kopf mit Signatur, „gib … zurück“, Notation nach EH §5 (←, solange, Verzweigung) • Hilfsschlange erzeugen; Warteliste vollständig entnehmen, Leerabfrage in der Schleifenbedingung • Namensvergleich; nur das vorderste passende Mitglied wird ausgelassen (Wahrheitswert gefunden) • übrige Mitglieder in die Hilfsschlange und anschließend vollständig zurück – Reihenfolge bleibt • Rückgabe wahr/falsch entsprechend • Randfälle (leere Warteliste, Name mehrfach, Name fehlt) korrekt, kein Zugriff auf leere Schlange	1	4	1	
e) Implementierung: Elemente aus einer dynamischen Reihung entfernen • Methodenkopf int ausschrauben(String sektor); Zähler und Rückgabe • Durchlauf mit gültigen Indizes 0 … getLength() – 1 (aktuelle Länge); Cast (Route) • Vergleich getSektor().equals(sektor), Entfernen mit delete(i) • Zählen der entfernten Routen • nach delete wird i nicht erhöht (bzw. Rückwärtsdurchlauf) – keine Route übersprungen, kein Indexfehler	1	3	1	
f) Fehleranalyse und Korrektur • Ablauf: beste = Morgenrot, keine Änderung für i = 1 … 6, getItem(7) → Laufzeitfehler • Fehler Schleifengrenze <= erläutert (ungültiger Index, bei jedem Aufruf) • Fehler Startwert erläutert (Route aus fremdem Sektor, kein null, leere Reihung, Index 0 ungeprüft) • Fehler == statt equals erläutert • vollständig korrigierte Operation (Startwert null mit Abfrage oder erste passende Route, Grenze, equals)	1	3	1	

Erwartete Prüfungsleistungen (Kernpunkte)	I	II	III	BE
g) Rekursionstabelle mit zweidimensionaler Reihung • Rekursionstiefen 0 bis 4 mit den Aufrufparametern (4, 2), (3, 1), (2, 2), (1, 3), (0, 3) • Werte wand[z][s]: 4, 3, 7, 6, 5 • ns jeweils korrekt, insbesondere bei Gleichstand (ns bleibt 3) und „–“ im Basisfall • Rückgabewerte von innen nach außen: 5, 11, 18, 21, 25	2	2		
h) Prinzip der Rekursion erläutern, Rekursionstiefe begründen • Prinzip: Selbstaufruf mit kleinerem Teilproblem (z − 1), Ergebnis = wand[z][s] + Ergebnis des Teilproblems • Basisfälle (z = 0, kein Griff darüber) und Terminierung, da z bei jedem Aufruf kleiner wird • Begründung d ≤ z über den Zeilenparameter z − d • Begründung „genau z ⟺ oberste Reihe erreicht“ einschließlich des vorzeitigen Abbruchs	1	2	1	
i) Tracetabelle eines Sortierverfahrens • Zeile i = 0 (pos = 1, Tausch Jana ↔ Kai) • Zeile i = 1 (pos = 5, Tausch Jana ↔ Ole) • Zeile i = 2 (pos = 2, kein Positionswechsel) • Zeile i = 3 (pos = 4: Nele, da 38 > 38 nicht gilt; Tausch Mo ↔ Nele) • Zeile i = 4 (pos = 5, Tausch Mo ↔ Jana); Endzustand	3	2		
j) Funktionsweise erläutern, Vergleiche zählen, Aufwand abschätzen • Verfahren als Auswahlsortierung (absteigend) erkannt, Grundidee • Erläuterung: Maximum im unsortierten Rest suchen, Tausch an Position i, sortierter Bereich wächst • Anzahl der Vergleiche für die Beispieldaten: 15 • allgemeine Anzahl n · (n − 1) / 2, unabhängig von den Daten • Abschätzung bei Verdopplung: etwa vervierfacht (quadratisch), begründet	1	3	1	
k) Stabilität untersuchen • Nachweis am Beispiel: Jana/Nele (38) vorher Jana vorn, nachher Nele vorn; Ursache Tausch bei i = 1 • Verallgemeinerung: Tausch über große Distanz → nicht stabil			2	
l) Einfügen in einen binären Suchbaum, Traversierung • Pia als linkes Kind von Sam (55) • Uwe als rechtes Kind von Rae (71) • Reihenfolge rechts – Wurzel – links (umgekehrte Inorder)	2	1		
m) Implementierung: rekursiv auf einem binären Suchbaum • Methodenkopf Mitglied naechstesZiel(BinTree b, int punkte); Rückgabe in jedem Pfad • Basisfall: leerer Baum → null, vor jedem getItem • Cast (Mitglied) b.getItem() und Vergleich über getPunkte() • Punktzahl der Wurzel ≤ punkte: Ergebnis der Suche im rechten Teilbaum zurückgeben • Punktzahl der Wurzel > punkte: linken Teilbaum rekursiv durchsuchen • Kandidatenlogik: liefert der linke Teilbaum null, ist die Wurzel das Ziel, sonst das Ergebnis von links • Ordnung genutzt (je Schritt nur ein Teilbaum) und Grenzfall gleiche Punktzahl korrekt (streng größer)	1	4	2	
Summe	18	32	10	**/ 60**

Die vom Prüfling gewählten Lösungsansätze und -wege müssen nicht mit denen in den dargestellten Lösungen identisch sein. Fachlich richtige Alternativen sind unter Berücksichtigung der verbindlichen Vorgaben (Ergänzende Hinweise, Neufassung 2025) entsprechend zu bewerten. Laufzeitfehler, die nicht durch explizite Abfragen ausgeschlossen werden, führen zu Abzug (EH §4).

Notenschlüssel

	15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
ab Prozent	95	90	85	80	75	70	65	60	55	50	45	40	33	27	20	0
ab BE (von 60)	57	54	51	48	45	42	39	36	33	30	27	24	20	17	12	0

Prozentgrenzen wie im niedersächsischen Zentralabitur (Erwartungshorizonte Informatik); BE-Grenzen auf ganze BE aufgerundet. **An die Schulvorgabe anpassen** (Fachkonferenzbeschluss zur Bewertung von Klausuren in der Qualifikationsphase).

Übersicht: Anforderungsbereiche und Kompetenzen

Teil	Inhalt / Kompetenz	Operator(en)	I	II	III	BE
a)	Klasse erläutern (Konstruktoren, Sichtbarkeit)	erläutern	3	1		4
b)	Klassendiagramm ergänzen (Vererbung)	ergänzen, begründen	1	2		3
c)	Stapel auswerten, Struktur unverändert	implementieren	1	5	1	7
d)	Struktogramm entwickeln (Schlange)	entwickeln	1	4	1	6
e)	Implementieren auf dynamischer Reihung	implementieren	1	3	1	5
f)	Fehler finden und korrigieren	analysieren, erläutern	1	3	1	5
g)	Rekursionstabelle, 2D-Reihung	dokumentieren	2	2		4
h)	Rekursion erläutern und begründen	erläutern, begründen	1	2	1	4
i)	Trace (Tracetabelle zur Vervollständigung)	dokumentieren	3	2		5
j)	Sortierverfahren analysieren, Effizienz durch Zählen	erläutern, bestimmen, abschätzen	1	3	1	5
k)	Stabilität untersuchen	untersuchen			2	2
l)	BST einfügen, Traversierung	zeichnen, angeben	2	1		3
m)	BST rekursiv implementieren	implementieren	1	4	2	7
Σ			18	32	10	60
%		Abitur-Richtwert 30 / 53 / 17 %	30	53	17	100

Rollen der Datenstrukturen in dieser Nachschreibklausur: **Stapel** als Implementierung mit unveränderter Struktur (c), **Schlange** im Struktogramm (d), **dynamische Reihung** als Implementierung und Fehleranalyse (e, f), **2D-Reihung** in der Rekursionstabelle (g, h), **statische Reihung** beim Sortieren mit Trace und Effizienz (i–k), **binärer Suchbaum** zeichnen und rekursiv implementieren (l, m). Zeitbudget: 60 BE · 2,5 min = 150 min + 30 min Einlesen und Kontrolle.

Korrekturhinweise

Allgemein

- Bewertet wird nach Kernpunkten; jeder Kernpunkt ist vollständig oder gar nicht erreicht (halbe BE nur nach Fachschaftsabsprache). Folgefehler werden nicht erneut abgezogen (insbesondere g → h und i → k).
- Java-Implementierungen: Kleine Syntaxfehler (fehlendes Semikolon, Klammer) führen nicht zu Abzug, solange die Logik eindeutig ist. Fehlende Casts (P4) und == statt equals (P5) sind fachliche Fehler: je höchstens 1 BE Abzug pro Aufgabenteil.
- Laufzeitfehler (P1): Fehlt eine Leer- oder Indexabfrage, wird der Kernpunkt „Randfälle/ohne Laufzeitfehler“ nicht vergeben – auch wenn die Beispielwerte stimmen.
- Struktogramme (d): Notation nach EH §5 ist Kernpunkt 1; ein Struktogramm mit Java-Schreibweisen wird inhaltlich voll bewertet, verliert aber diesen Kernpunkt (P9).
- Zweifelsfälle bei c), e), f) und m) mit dem Papier-Compiler entscheiden (AQ_W6_Nachschreibklausur_Papier-Compiler.txt, nur Lehrkraft): Handschrift unverändert abtippen, Testrunner starten. Die Tests ersetzen nicht die Bewertung nach Kernpunkten.

Top 10 Punktverluste – wo sie in dieser Klausur auftreten

Nr.	Punktverlust	Aufgabenteile
P1	Laufzeitfehler nicht ausgeschlossen	c (pop), d (dequeue), e/f (getItem), m (getItem auf leerem Baum)
P2	Struktur zerstört statt unverändert	c (Tourenbuch), d (Reihenfolge der Warteliste)
P3	Signatur verfehlt / Rückgabe fehlt	c, e, m (Methodenköpfe), d (gib … zurück)
P4	Typen und Casts	c (Route), e/f (Route), m (Mitglied)
P5	== statt equals	d (Namen), e (Sektor), f (Fehler 3)
P6	Off-by-one	c (anzahl < k), e (delete in Vorwärtsschleife), f (Fehler 1), m (streng größer: ≤ statt <)
P7	Rekursion: Basisfall, Ergebnis, Tiefe	g (Tiefe 0 beim Erstaufruf), h, m (Basisfall isEmpty, Ergebnis des linken Teilbaums verwenden)
P8	Trace-/Dokumentationstabelle	g (Rückgabewerte), i (Zeile ohne Tausch)
P9	Struktogramm nicht nach EH §5	d
P10	Operator verfehlt	a, b, f, h, j, k (erläutern/begründen/untersuchen)

Musterlösungen, Kernpunkte und typische Fehler

a) Konstruktoren und Sichtbarkeit erläutern 4 BE (I 3 · II 1 · III 0)

Musterlösung

Ein **Konstruktor** erzeugt ein neues Objekt der Klasse und belegt seine Attribute mit Anfangswerten. Die Klasse Mitglied besitzt zwei Konstruktoren mit gleichem Namen, aber unterschiedlichen Parameterlisten (Überladen); welcher ausgeführt wird, entscheidet sich an den beim Aufruf übergebenen Parametern.

c Mitglied(name: Zeichenkette) legt ein neues Mitglied mit dem übergebenen Namen an; es hat noch 0 Punkte und ein leeres Tourenbuch (neuer, leerer Stapel). **c Mitglied(name: Zeichenkette, punkte: Ganzzahl)** übernimmt zusätzlich eine Startpunktzahl, z. B. für ein Mitglied, das bereits Punkte gesammelt hat, oder für die Teilnehmenden eines Wettkampfs mit ihren Wettkampfpunkten (Abbildung 9).

– kennzeichnet private Attribute: Auf name, punkte und tourenbuch kann nur innerhalb der Klasse Mitglied zugegriffen werden. **+** kennzeichnet öffentliche Operationen, die von anderen Klassen aufgerufen werden können. So bleiben die Daten gekapselt: Die Punkte können z. B. von außen nur gelesen (getPunkte) und nur über klettern verändert werden, sodass Tourenbuch und Punktzahl zusammenpassen.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Konstruktor erzeugt ein Objekt und initialisiert die Attribute	I	1	☐
c Mitglied(name): Name gesetzt, 0 Punkte, leeres Tourenbuch	I	1	☐
c Mitglied(name, punkte): zusätzlich Startpunktzahl; Überladen, Auswahl über die Parameterliste, sinnvoller Anwendungsfall	II	1	☐
– privat (nur innerhalb der Klasse), + öffentlich; Kapselung (Zugriff nur über Operationen)	I	1	☐

Alternativen / Teilpunkte: Andere sinnvolle Anwendungsfälle für den zweiten Konstruktor (Datenübernahme, Wettkampf) sind gleichwertig. Eine bloße Nennung „es gibt zwei Konstruktoren“ ohne Funktion: kein Punkt für die Kernpunkte 2 und 3.

Typische Fehler

- Konstruktor als „Methode, die etwas zurückgibt“ beschrieben (P10).
- Nur Nennung statt Erläuterung (P10): Bezug zur Klasse Mitglied fehlt.
- +/– mit „wichtig/unwichtig“ oder „Getter/Setter“ verwechselt.

b) Klassendiagramm um die Klasse Boulder ergänzen 3 BE (I 1 · II 2 · III 0)

Musterlösung

Ergänzung (nur der relevante Ausschnitt; Notation nach EH §6):

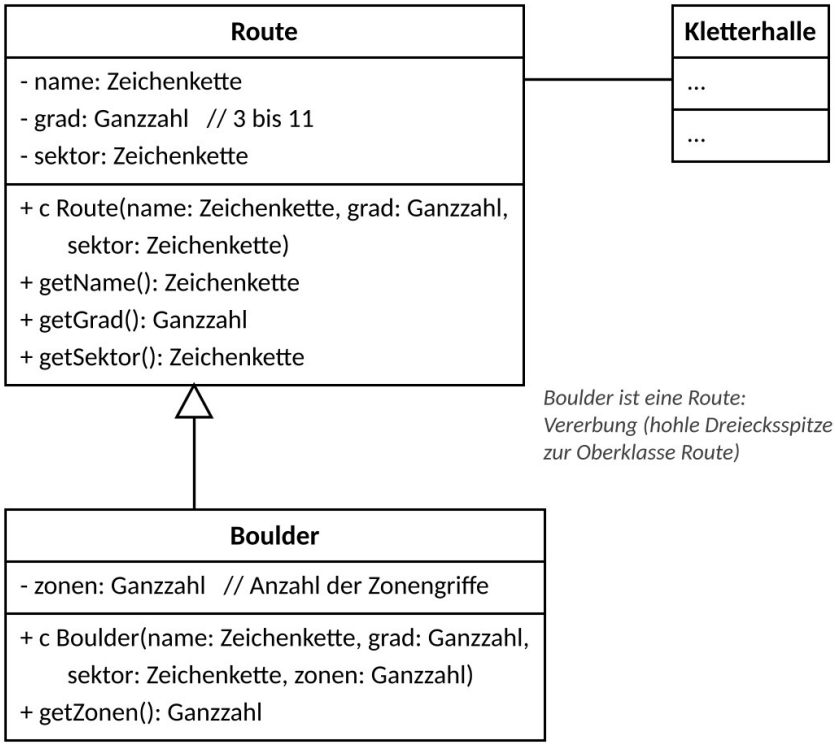


Abbildung EH.1: Klasse Boulder als Unterklasse von Route

Begründung: Ein Boulder **ist eine** Route mit einer zusätzlichen Eigenschaft. Durch **Vererbung** übernimmt Boulder Name, Schwierigkeitsgrad, Sektor und die zugehörigen Operationen von Route, ohne sie doppelt zu modellieren; ergänzt werden nur zonen und getZonen. Da jedes Boulder-Objekt auch vom Typ Route ist, kann es in der dynamischen Reihung routen (Inhaltstyp Route) gespeichert und von Operationen wie ausschrauben und schwersteRoute wie jede andere Route verarbeitet werden. Der Konstruktor von Boulder ruft dazu den Konstruktor der Oberklasse auf.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Klassenkarte Boulder mit – zonen: Ganzzahl, Konstruktor mit allen vier Parametern, + getZonen(): Ganzzahl	I	1	☐
Vererbungspfeil mit hohler Dreiecksspitze von Boulder zu Route	II	1	☐
Begründung: „ist ein“, Wiederverwendung der geerbten Attribute/Operationen, Speicherung in routen möglich	II	1	☐

Alternativen / Teilpunkte: Eine Assoziation Boulder – Route oder eine eigenständige Klasse mit allen Attributen ist fachlich schwächer (Doppelung, nicht in routen speicherbar): höchstens 1 BE (Klassenkarte). Fehlt im Konstruktor ein Parameter der Oberklasse: Abzug beim ersten Kernpunkt.

Typische Fehler

- Pfeilspitze an der Unterklasse oder gefüllte Spitze.
- Attribute name, grad, sektor in Boulder wiederholt (Vererbung nicht verstanden).
- Konstruktor ohne „c“ bzw. ohne Sichtbarkeit (EH §6).
- Begründung ohne Bezug auf die Reihung routen (P10).

c) Implementierung: Stapel auswerten und unverändert lassen 7 BE (I 1 · II 5 · III 1)

Musterlösung

Musterlösung (Java; mit javac 21 und den Tests des Papier-Compilers geprüft: 7/7 grün; Abbildung 2: k = 1 → 4, 2 → 6, 3 → 9, 4 → 9, 5 → 10, 10 → 10, 0 → 0, leeres Tourenbuch → 0; Tourenbuch danach unverändert):

```
public int hoechsterGrad(int k) {
    Stack hilf = new Stack();
    int max = 0;
    int anzahl = 0;
    while (!tourenbuch.isEmpty() && anzahl < k) {
        Route r = (Route) tourenbuch.pop();
        if (r.getGrad() > max) {
            max = r.getGrad();
        }
        hilf.push(r);
        anzahl = anzahl + 1;
    }
    while (!hilf.isEmpty()) {
        tourenbuch.push(hilf.pop());
    }
    return max;
}
```

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Methodenkopf int hoechsterGrad(int k); Rückgabe in jedem Pfad	I	1	☐
Hilfsstapel; Entnehmen mit pop und Cast (Route)	II	1	☐
Schleife endet nach k Routen oder bei leerem Tourenbuch; Leerabfrage vor jedem pop	II	1	☐
Maximum über getGrad() mit geeignetem Startwert 0	II	1	☐
vollständiges Zurückschichten aus dem Hilfsstapel – Tourenbuch unverändert	II	1	☐
Rückgabe des Maximums, 0 bei leerem Tourenbuch bzw. k ≤ 0	II	1	☐
Randfälle (k größer als die Anzahl, k = 0, leer) ohne Laufzeitfehler; keine Rückgabe vor dem Zurückschichten	III	1	☐

Alternativen / Teilpunkte: Gleichwertig (beide ebenfalls 7/7 grün): Zählschleife `for (int i = 0; i < k && !tourenbuch.isEmpty(); i++)`; oder den ganzen Stapel umschichten und nur die ersten k Routen auswerten. Ein Startwert über top() ist nur mit vorheriger Leerabfrage richtig. Zurückschichten über eine Schlange kehrt die Reihenfolge um (Test rot): Kernpunkt 5 nicht erreicht. Fehlender Cast oder Kleinigkeiten der Syntax: höchstens 1 BE Abzug insgesamt.

Alternative: Zählschleife mit Leerabfrage in der Bedingung

```
public int hoechsterGrad(int k) {
    Stack hilf = new Stack();
    int max = 0;
    for (int i = 0; i < k && !tourenbuch.isEmpty(); i++) {
        Route r = (Route) tourenbuch.pop();
        hilf.push(r);
        if (r.getGrad() > max) {
            max = r.getGrad();
        }
    }
    while (!hilf.isEmpty()) {
        tourenbuch.push(hilf.pop());
    }
    return max;
}
```

Typische Fehler

- while (anzahl < k) ohne Leerabfrage → pop auf leerem Stapel bei k > 5 bzw. leerem Tourenbuch (P1).
- Tourenbuch nicht wiederhergestellt oder `tourenbuch = hilf;` – Reihenfolge umgekehrt (P2).
- return im Schleifenrumpf, bevor zurückgeschichtet wurde (P2/P3).
- anzahl <= k: eine Route zu viel betrachtet (P6).
- (Route) vergessen – Object hat kein getGrad (P4).

d) Struktogramm: aus einer Schlange entfernen, Reihenfolge erhalten 6 BE (I 1 · II 4 · III 1)

Musterlösung

Musterlösung (nach EH §5; die zeilengetreue Java-Übersetzung ist mit den Tests des Papier-Compilers geprüft: 5/5 grün; abmelden("Ada") → wahr, danach Ben, Cem, Ada, Dana; abmelden("Eva") → falsch, Warteliste unverändert):

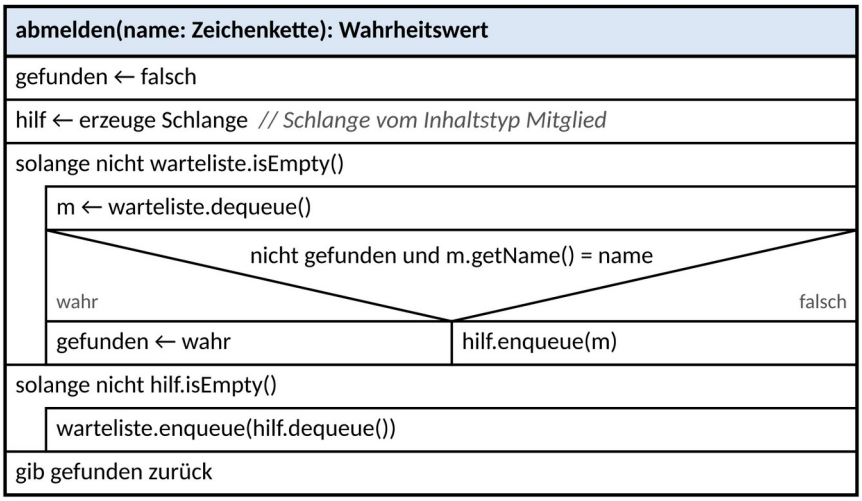


Abbildung EH.2: Struktogramm der Operation abmelden

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Kopf mit Signatur, „gib … zurück“, Notation nach EH §5 (←, solange, Verzweigung)	I	1	☐
Hilfsschlange erzeugen; Warteliste vollständig entnehmen, Leerabfrage in der Schleifenbedingung	II	1	☐
Namensvergleich; nur das vorderste passende Mitglied wird ausgelassen (Wahrheitswert gefunden)	II	1	☐

Kernpunkt	AFB	BE	erreicht
übrige Mitglieder in die Hilfsschlange und anschließend vollständig zurück – Reihenfolge bleibt	II	1	☐
Rückgabe wahr/falsch entsprechend	II	1	☐
Randfälle (leere Warteliste, Name mehrfach, Name fehlt) korrekt, kein Zugriff auf leere Schlange	III	1	☐

Alternativen / Teilpunkte: Gleichwertig: Rotieren innerhalb der Warteliste mit vorher bestimmter Anzahl (Zählvariable) oder Umschichten in eine Hilfsschlange und zurück mit Auslassen beim Zurückschichten (geprüft, 5/5). Ein Abbruch beim Treffer ist nur richtig, wenn danach der Rest der Warteliste und die Hilfsschlange korrekt zusammengeführt werden. Java-Code statt Struktogramm: höchstens 3 BE; einzelne Java-Schreibweisen (==, ;, ++) im Struktogramm: 1 BE Abzug (P9).

Typische Fehler

- Alle Mitglieder mit dem Namen entfernt (Merker gefunden fehlt).
- Zurückschichten über einen Stapel → Reihenfolge umgekehrt (P2).
- Schleife „solange warteliste.head().getName() ≠ name“ – head auf leerer Schlange, wenn der Name fehlt (P1).
- Java-Syntax statt EH-Notation, Rückgabe fehlt (P9, P3).

e) Implementierung: Elemente aus einer dynamischen Reihung entfernen 5 BE (I 1 · II 3 · III 1)

Musterlösung

Musterlösung (Java; Papier-Compiler 5/5 grün; Abbildung 4: „Dach“ → 4, danach Morgenrot, Sandmann, Wolkenkratzer; „Platte“ → 2; „Kante“ → 1; „Turm“ → 0; leere Reihung → 0):

```
public int ausschrauben(String sektor) {
    int anzahl = 0;
    int i = 0;
    while (i < routen.getLength()) {
        Route r = (Route) routen.getItem(i);
        if (r.getSektor().equals(sektor)) {
            routen.delete(i);
            anzahl = anzahl + 1;
        } else {
            i = i + 1;
        }
    }
    return anzahl;
}
```

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Methodenkopf int ausschrauben(String sektor); Zähler und Rückgabe	I	1	☐
Durchlauf mit gültigen Indizes 0 … getLength() – 1 (aktuelle Länge); Cast (Route)	II	1	☐
Vergleich getSektor().equals(sektor), Entfernen mit delete(i)	II	1	☐
Zählen der entfernten Routen	II	1	☐
nach delete wird i nicht erhöht (bzw. Rückwärtsdurchlauf) – keine Route übersprungen, kein Indexfehler	III	1	☐

Alternativen / Teilpunkte: Gleichwertig (geprüft, 5/5): Rückwärtsschleife `for (int i = routen.getLength() - 1; i >= 0; i--)` – hier bleibt die Reihenfolge der übrigen Routen ebenfalls erhalten; Zählschleife mit `i--` nach delete. Vorwärtsschleife mit festem i++: Schwalbennest wird gelöscht, Dachrinne rückt nach und wird übersprungen

(ausschrauben("Dach") liefert 3) – Kernpunkt 5 nicht erreicht. Länge vor der Schleife in einer Variablen festgehalten: zusätzlich Indexfehler (P1).

Alternative: Rückwärtsdurchlauf

```
public int ausschrauben(String sektor) {
    int anzahl = 0;
    for (int i = routen.getLength() - 1; i >= 0; i--) {
        if (((Route) routen.getItem(i)).getSektor().equals(sektor)) {
            routen.delete(i);
            anzahl++;
        }
    }
    return anzahl;
}
```

Typische Fehler

- delete in Vorwärtsschleife mit i++ → benachbarte Routen übersprungen (P6).
- n = routen.getLength() vor der Schleife → getItem mit ungültigem Index (P1/P6).
- getSektor() == sektor (P5).
- Nur gezählt, nicht entfernt (Operator „entfernt“ überlesen).

f) Fehleranalyse und Korrektur 5 BE (I 1 · II 3 · III 1)

Musterlösung

Ablauf von schwersteRoute("Kante") mit der Implementierung aus Abbildung 5 (durch Ausführen der fehlerhaften Fassung geprüft): Vor der Schleife ist beste = Morgenrot (Grad 8, Sektor Platte), obwohl diese Route nicht im Sektor Kante liegt.

i	r = routen.getItem(i)	beste danach
1	Kalkfresser (7, Dach)	Morgenrot
2	Sandmann (4, Platte)	Morgenrot
3	Schwalbennest (6, Dach)	Morgenrot
4	Dachrinne (9, Dach)	Morgenrot
5	Wolkenkratzer (6, Kante)	Morgenrot
6	Spinne (10, Dach)	Morgenrot
7	getItem(7) bei Länge 7	Laufzeitfehler: IndexOutOfBoundsException, Abbruch

Fehler 1 – Schleifengrenze (Zeile 3): `i <= routen.getLength()` lässt i bis zur Länge 7 laufen; `getItem(7)` greift auf einen nicht existierenden Index zu → Laufzeitfehler (IndexOutOfBoundsException). Er tritt bei **jedem** Aufruf mit nicht leerer Reihung auf. Richtig: `i < routen.getLength()`.

Fehler 2 – Startwert (Zeile 2): beste wird unabhängig vom Sektor mit der ersten Route belegt. Liegt diese Route nicht im gesuchten Sektor und ist sie schwerer als alle Routen des Sektors, wird eine falsche Route geliefert (hier Morgenrot statt Wolkenkratzer); null kann nie zurückgegeben werden; bei leerer Reihung führt getItem(0) zum Laufzeitfehler. Außerdem wird Index 0 in der Schleife nie auf den Sektor geprüft.

Fehler 3 – Vergleich (Zeile 5): Zeichenketten werden mit == verglichen; das prüft in Java die Identität der Objekte, nicht den Inhalt. Richtig: `r.getSektor().equals(sektor)`.

Korrektur (Papier-Compiler 5/5 grün: Kante → Wolkenkratzer, Dach → Spinne, Platte → Morgenrot, Turm → null, leere Reihung → null):

```
public Route schwersteRoute(String sektor) {
    Route beste = null;
    for (int i = 0; i < routen.getLength(); i++) {
        Route r = (Route) routen.getItem(i);
        if (r.getSektor().equals(sektor)) {
            if (beste == null || r.getGrad() > beste.getGrad()) {
                beste = r;
            }
        }
    }
    return beste;
}
```

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Ablauf: beste = Morgenrot, keine Änderung für i = 1 … 6, getItem(7) → Laufzeitfehler	I	1	☐
Fehler Schleifengrenze <= erläutert (ungültiger Index, bei jedem Aufruf)	II	1	☐
Fehler Startwert erläutert (Route aus fremdem Sektor, kein null, leere Reihung, Index 0 ungeprüft)	II	1	☐
Fehler == statt equals erläutert	II	1	☐
vollständig korrigierte Operation (Startwert null mit Abfrage oder erste passende Route, Grenze, equals)	III	1	☐

Alternativen / Teilpunkte: Gleichwertige Korrektur (geprüft, 5/5): zuerst den Index der ersten Route des Sektors suchen, null liefern, wenn es keine gibt, danach ab dort das Maximum bestimmen. Nur Grenze und equals korrigiert, Startwert unverändert: Kernpunkt 5 nicht erreicht (Kante liefert weiter Morgenrot, leere Reihung stürzt ab). Startwert null ohne Abfrage `beste == null` vor beste.getGrad(): Laufzeitfehler (NullPointerException), Kernpunkt 5 nicht erreicht.

Typische Fehler

- Nur den Absturz erkannt, den falschen Startwert übersehen (P10: Analyse unvollständig).
- Korrektur `i < getLength() - 1` (letzte Route Spinne nie geprüft, P6).
- Startwert null ohne null-Abfrage → NullPointerException (P1).
- == als „in der Online-IDE funktioniert es“ akzeptiert – im Abitur ist equals verlangt (P5).

g) Rekursionstabelle mit zweidimensionaler Reihung 4 BE (I 2 · II 2 · III 0)

Musterlösung

Dokumentation (durch Ausführen der zeilengetreuen Java-Fassung von Abbildung 7 und eines unabhängigen Python-Modells geprüft):

Rekursionstiefe	z	s	wand[z][s]	ns	Rückgabewert
0 (Erstaufruf)	4	2	4	1	25
1	3	1	3	2	21
2	2	2	7	3	18
3	1	3	6	3	11
4	0	3	5	–	5

Der Weg führt über (4, 2) → (3, 1) → (2, 2) → (1, 3) → (0, 3). In Zeile 1, Spalte 3 sind die Griffe schräg links darüber (wand[0][2] = 5) und direkt darüber (wand[0][3] = 5) gleich gut; wegen des Vergleichs mit > bleibt ns = 3. Rückgabe des Erstaufrufs: **25**.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Rekursionstiefen 0 bis 4 mit den Aufrufparametern (4, 2), (3, 1), (2, 2), (1, 3), (0, 3)	I	1	☐
Werte wand[z][s]: 4, 3, 7, 6, 5	I	1	☐
ns jeweils korrekt, insbesondere bei Gleichstand (ns bleibt 3) und „–“ im Basisfall	II	1	☐
Rückgabewerte von innen nach außen: 5, 11, 18, 21, 25	II	1	☐

Alternativen / Teilpunkte: Folgefehler: Ist ns in einer Zeile falsch, werden die folgenden Zeilen auf Grundlage des falschen Wegs bewertet. Wird bei Gleichstand nach links gegangen (ns = 2), ergibt sich derselbe Rückgabewert, aber Kernpunkt 3 wird nicht vergeben. Rekursionstiefe beim Erstaufruf 1 statt 0: 1 BE Abzug (P7).

Typische Fehler

- Rekursionstiefe ab 1 gezählt (P7).
- Rückgabewerte in der Reihenfolge der Aufrufe statt von innen nach außen eingetragen (P8).
- Gleichstand bei (1, 3) falsch aufgelöst (> statt ≥ nicht beachtet).
- Zeile und Spalte vertauscht (wand[s][z]).

h) Prinzip der Rekursion erläutern, Rekursionstiefe begründen 4 BE (I 1 · II 2 · III 1)

Musterlösung

Prinzip: Bei einer Rekursion ruft sich eine Operation selbst mit einem kleineren, gleichartigen Teilproblem auf. aufstieg(z, s) löst „Summe des Weges ab Griff (z, s)“, indem es den besten Griff (z − 1, ns) in der Reihe darüber bestimmt und das Teilproblem aufstieg(z − 1, ns) – den Weg ab diesem Griff – rekursiv löst. Das Ergebnis setzt sich aus der eigenen Griffqualität wand[z][s] und dem Ergebnis des rekursiven Aufrufs zusammen.

Abbruch (Basisfälle): Die Rekursion endet, wenn die oberste Reihe erreicht ist (z = 0) oder wenn in der Reihe darüber kein erreichbarer Griff existiert (wand[z − 1][ns] = 0); dann wird nur wand[z][s] zurückgegeben. Da z bei jedem Aufruf um 1 kleiner wird, wird spätestens bei z = 0 ein Basisfall erreicht – die Rekursion terminiert.

Begründung Rekursionstiefe: Der Erstaufruf hat die Tiefe 0 und den Zeilenparameter z; ein Aufruf in Tiefe d hat den Zeilenparameter z − d, weil jeder rekursive Aufruf die Zeile um genau 1 verringert. Rekursive Aufrufe finden nur für Zeilenparameter > 0 statt, also gilt z − d ≥ 0 bzw. d ≤ z. Die Tiefe z wird genau dann erreicht, wenn ein Aufruf mit Zeilenparameter 0 stattfindet, also wenn der Weg die oberste Reihe erreicht. Endet der Weg vorher, weil über einem Griff kein Griff mehr erreichbar ist, bricht die Rekursion bei einem Zeilenparameter > 0 ab und die Tiefe ist kleiner als z. Beispiel: aufstieg(4, 2) erreicht Zeile 0 mit Tiefe 4.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Prinzip: Selbstaufruf mit kleinerem Teilproblem (z − 1), Ergebnis = wand[z][s] + Ergebnis des Teilproblems	I	1	☐
Basisfälle (z = 0, kein Griff darüber) und Terminierung, da z bei jedem Aufruf kleiner wird	II	1	☐
Begründung d ≤ z über den Zeilenparameter z − d	II	1	☐
Begründung „genau z ⇔ oberste Reihe erreicht“ einschließlich des vorzeitigen Abbruchs	III	1	☐

Alternativen / Teilpunkte: Eine Argumentation über die Zahl der Reihen zwischen Start und Ziel ist gleichwertig. Allgemeine Definition der Rekursion ohne Bezug zur Operation aufstieg: höchstens 1 BE für die ersten beiden Kernpunkte (P10).

Typische Fehler

- Nur „die Methode ruft sich selbst auf“ – Teilproblem und Zusammensetzung des Ergebnisses fehlen (P10).
- Zweiten Basisfall (kein Griff) übersehen.
- „Rekursionstiefe = Anzahl der Aufrufe“ (Erstaufruf mitgezählt, P7).
- „begründen“ durch ein Beispiel ersetzt (P10).

i) Tracetabelle eines Sortierverfahrens 5 BE (I 3 · II 2 · III 0)

Musterlösung

Tracetabelle (durch Ausführen der zeilengetreuen Java-Fassung von Abbildung 10 mit Protokollausgabe geprüft):

i	pos	t[0]	t[1]	t[2]	t[3]	t[4]	t[5]
Start	–	Jana 38	Kai 57	Lea 47	Mo 31	Nele 38	Ole 52
0	1	Kai 57	Jana 38	Lea 47	Mo 31	Nele 38	Ole 52
1	5	Kai 57	Ole 52	Lea 47	Mo 31	Nele 38	Jana 38
2	2	Kai 57	Ole 52	Lea 47	Mo 31	Nele 38	Jana 38
3	4	Kai 57	Ole 52	Lea 47	Nele 38	Mo 31	Jana 38
4	5	Kai 57	Ole 52	Lea 47	Nele 38	Jana 38	Mo 31

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Zeile i = 0 (pos = 1, Tausch Jana ↔ Kai)	I	1	☐
Zeile i = 1 (pos = 5, Tausch Jana ↔ Ole)	I	1	☐
Zeile i = 2 (pos = 2, kein Positionswechsel)	I	1	☐
Zeile i = 3 (pos = 4: Nele, da 38 > 38 nicht gilt; Tausch Mo ↔ Nele)	II	1	☐
Zeile i = 4 (pos = 5, Tausch Mo ↔ Jana); Endzustand	II	1	☐

Alternativen / Teilpunkte: Folgefehler werden nicht erneut abgezogen. Werden nur die Namen ohne Punkte eingetragen, ist das gleichwertig, sofern eindeutig. Eine zusätzliche Zeile je Durchlauf der inneren Schleife ist nicht verlangt, aber nicht falsch, wenn die Zeilen an der Protokollierungsstelle erkennbar sind (P8).

Typische Fehler

- Bei i = 2 eine Zeile ausgelassen, weil „nichts passiert“ (P8).
- Bei i = 3 pos = 5 (Jana) – Vergleich mit > nicht beachtet.
- Tausch nicht ausgeführt, nur pos notiert.
- Aufsteigend statt absteigend sortiert.

j) Funktionsweise erläutern, Vergleiche zählen, Aufwand abschätzen 5 BE (I 1 · II 3 · III 1)

Musterlösung

Funktionsweise: sortiere ist ein **Sortieren durch Auswahl** (Selectionsort), hier absteigend. In jedem Durchlauf der äußeren Schleife sucht die innere Schleife im noch unsortierten Bereich t[i] … t[n − 1] die Position pos des Mitglieds mit den meisten Punkten; anschließend werden t[i] und t[pos] getauscht. Danach steht an Position i das Mitglied mit der i-größten Punktzahl, der Bereich t[0] … t[i] ist endgültig sortiert. Nach n − 1 Durchläufen ist auch das letzte Element an der richtigen Stelle.

Anzahl der Vergleiche: Für i = 0, 1, …, 4 vergleicht die innere Schleife n − 1 − i Punktzahlen: 5 + 4 + 3 + 2 + 1 = **15** Vergleiche (durch Zählen im Programm bestätigt). Die Anzahl hängt nicht von den Daten ab; allgemein sind es (n − 1) + … + 1 = n · (n − 1) / 2 Vergleiche.

Abschätzung bei Verdopplung: Für 2n Teilnehmende sind es 2n · (2n − 1) / 2 = n · (2n − 1) Vergleiche, also für große n etwa das **Vierfache** (quadratisches Wachstum), z. B. 6 → 15, 12 → 66 (≈ 4,4-fach), 1000 → 499500, 2000 → 1999000 (≈ 4,0-fach).

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Verfahren als Auswahlsortierung (absteigend) erkannt, Grundidee	I	1	☐
Erläuterung: Maximum im unsortierten Rest suchen, Tausch an Position i, sortierter Bereich wächst	II	1	☐
Anzahl der Vergleiche für die Beispieldaten: 15	II	1	☐
allgemeine Anzahl n · (n − 1) / 2, unabhängig von den Daten	II	1	☐
Abschätzung bei Verdopplung: etwa vervierfacht (quadratisch), begründet	III	1	☐

Alternativen / Teilpunkte: Die Abschätzung kann auch über die Struktur der Schleifen begründet werden (doppelt so viele äußere Durchläufe mit je etwa doppelt so vielen inneren Vergleichen). „Doppelt so viele Vergleiche“ ohne Begründung: Kernpunkt 5 nicht erreicht. Ohne Fachbegriff, aber mit korrekter Beschreibung: voll für Kernpunkt 2, Kernpunkt 1 nur bei erkennbarer Grundidee.

Typische Fehler

- Anzahl der Tauschvorgänge statt der Vergleiche gezählt.
- $n^2 = 36$ Vergleiche angegeben.
- „abschätzen“ ohne Begründung (P10).
- Funktionsweise als Befehlsfolge nacherzählt statt erläutert (P10).

k) Stabilität untersuchen 2 BE (I 0 · II 0 · III 2)

Musterlösung

Die Reihenfolge gleicher Punktzahlen bleibt **nicht** erhalten: Jana und Nele haben beide 38 Punkte; bei der Anmeldung steht Jana (Index 0) vor Nele (Index 4), nach dem Sortieren steht Nele (Index 3) vor Jana (Index 4). Ursache ist der Tausch im Durchlauf i = 1: Jana wird mit Ole getauscht und springt dabei von Index 1 auf Index 5 – über Nele hinweg. Allgemein kann ein Tausch über eine große Distanz ein Element hinter ein gleichwertiges Element verschieben; das Verfahren ist deshalb nicht stabil. (Für eine faire Siegerehrung bei Punktgleichheit müsste z. B. eine zweite Ordnung festgelegt werden.)

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Nachweis am Beispiel: Jana/Nele (38) vorher Jana vorn, nachher Nele vorn; Ursache Tausch bei i = 1	III	1	☐
Verallgemeinerung: Tausch über große Distanz → nicht stabil	III	1	☐

Alternativen / Teilpunkte: Auch ein anderer korrekter Nachweis aus der eigenen (folgerichtigen) Tracetabelle wird anerkannt. Die bloße Aussage „nicht stabil“ ohne Nachweis: 0 BE.

Typische Fehler

- Urteil „stabil“, weil Jana und Nele am Ende benachbart sind.
- Behauptung ohne Bezug zur Tracetabelle (P10).

I) Einfügen in einen binären Suchbaum, Traversierung 3 BE (I 2 · II 1 · III 0)

Musterlösung

Pia (50): 50 < 60 → links zu Ida (35); 50 > 35 → rechts zu Lou (48); 50 > 48 → rechts zu Sam (55); 50 < 55 → linkes Kind von Sam. Uwe (77): 77 > 60 → rechts zu Vera (82); 77 < 82 → links zu Rae (71); 77 > 71 → rechtes Kind von Rae.

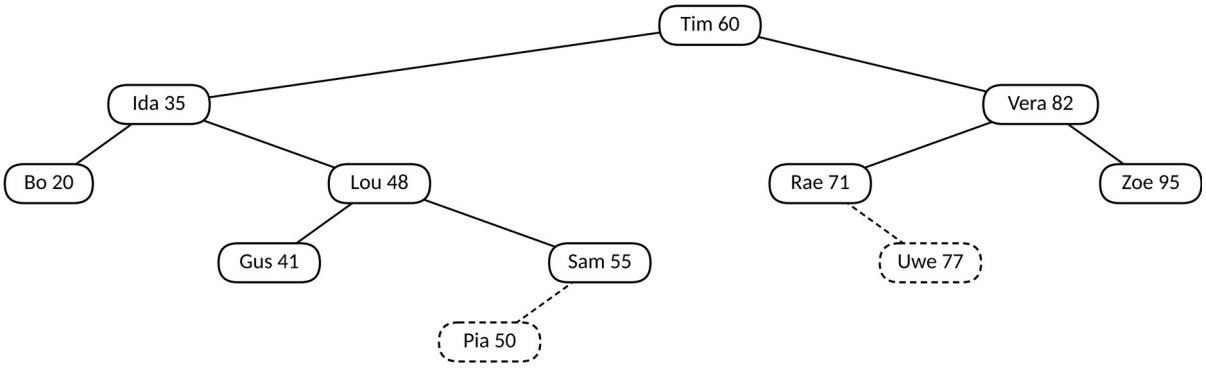


Abbildung EH.3: Bestenliste nach dem Einfügen von Pia und Uwe (neue Knoten gestrichelt)

Absteigende Ausgabe: **umgekehrte Inorder-Traversierung** – zuerst den rechten Teilbaum, dann die Wurzel, dann den linken Teilbaum (jeweils rekursiv). Ausgabe: Zoe 95, Vera 82, Uwe 77, Rae 71, Tim 60, Sam 55, Pia 50, Lou 48, Gus 41, Ida 35, Bo 20.

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Pia als linkes Kind von Sam (55)	I	1	☐
Uwe als rechtes Kind von Rae (71)	I	1	☐
Reihenfolge rechts – Wurzel – links (umgekehrte Inorder)	II	1	☐

Alternativen / Teilpunkte: „Inorder und das Ergebnis umdrehen“ ist gleichwertig. „Inorder“ allein liefert eine aufsteigende Ausgabe: Kernpunkt 3 nicht erreicht.

Typische Fehler

- Neuer Knoten an einer inneren Position eingefügt (Baum umgebaut).
- Pia als rechtes Kind von Lou (48) – Sam übersehen.
- Preorder oder Level-Order genannt.

m) Implementierung: rekursiv auf einem binären Suchbaum 7 BE (I 1 · II 4 · III 2)

Musterlösung

Musterlösung (Java; Papier-Compiler 7/7 grün; Abbildung 12: 50 → Sam, 60 → Rae, 48 → Sam, 75 → Vera, 20 → Ida, 0 → Bo, 95 → null, leerer Baum → null; bei punkte = 50 werden nur Tim, Ida, Lou, Sam untersucht). Idee: Hat die Wurzel höchstens punkte Punkte, kann das Ziel nur im rechten Teilbaum liegen. Hat sie mehr, ist sie ein Kandidat; ein besseres Ziel (kleinere Punktzahl, aber noch größer als punkte) kann nur im linken Teilbaum liegen – gibt es dort keines, ist die Wurzel selbst das Ziel.

```
public Mitglied naechstesZiel(BinTree b, int punkte) {
    if (b.isEmpty()) {
        return null;
    }
    Mitglied m = (Mitglied) b.getItem();
    if (m.getPunkte() <= punkte) {
        return naechstesZiel(b.getRight(), punkte);
    }
    Mitglied links = naechstesZiel(b.getLeft(), punkte);
    if (links == null) {
        return m;
    }
    return links;
}
```

Kernpunkte und BE-Verteilung

Kernpunkt	AFB	BE	erreicht
Methodenkopf Mitglied naechstesZiel(BinTree b, int punkte); Rückgabe in jedem Pfad	I	1	☐
Basisfall: leerer Baum → null, vor jedem getItem	II	1	☐
Cast (Mitglied) b.getItem() und Vergleich über getPunkte()	II	1	☐
Punktzahl der Wurzel ≤ punkte: Ergebnis der Suche im rechten Teilbaum zurückgeben	II	1	☐
Punktzahl der Wurzel > punkte: linken Teilbaum rekursiv durchsuchen	II	1	☐
Kandidatenlogik: liefert der linke Teilbaum null, ist die Wurzel das Ziel, sonst das Ergebnis von links	III	1	☐
Ordnung genutzt (je Schritt nur ein Teilbaum) und Grenzfall gleiche Punktzahl korrekt (streng größer)	III	1	☐

Alternativen / Teilpunkte: Gleichwertig (geprüft, 7/7): Hilfsoperation mit dem bisher besten Kandidaten als Parameter (`suche(b, punkte, kandidat)`). Eine iterative Lösung mit Schleife besteht zwar alle Tests, erfüllt aber die Vorgabe „rekursiv“ nicht: höchstens 5 BE. Eine Lösung, die den **ganzen Baum** durchsucht und das Minimum aller größeren Punktzahlen bestimmt, liefert richtige Ergebnisse (Papier-Compiler 6/7, nur m_nutztSuchbaumOrdnung rot): Kernpunkt 7 nicht erreicht, alle anderen möglich. Basisfall über isLeaf statt isEmpty: Laufzeitfehler bei Knoten mit nur einem Kind (P1/P7).

Alternative: Kandidat als Parameter einer Hilfsoperation

```
public Mitglied naechstesZiel(BinTree b, int punkte) {
    return suche(b, punkte, null);
}

private Mitglied suche(BinTree b, int punkte, Mitglied kandidat) {
    if (b.isEmpty()) {
        return kandidat;
    }
    Mitglied m = (Mitglied) b.getItem();
    if (m.getPunkte() > punkte) {
        return suche(b.getLeft(), punkte, m);
    } else {
        return suche(b.getRight(), punkte, kandidat);
    }
}
```

Typische Fehler

- Basisfall fehlt bzw. getItem vor isEmpty (P1/P7).
- Ergebnis des linken Teilbaums ohne null-Prüfung zurückgegeben – für 50 käme null statt Sam (P7).
- Ergebnis des rekursiven Aufrufs nicht verwendet, immer die Wurzel geliefert (P7).
- < statt ≤ beim Vergleich: Lou mit genau 48 Punkten als Ziel für 48 (P6).
- Richtung im Suchbaum vertauscht.
- (Mitglied) vergessen (P4).

Anhang: Prüfung der Musterlösungen und Testklasse (Papier-Compiler)

Alle Java-Musterlösungen, die Java-Übersetzung des Struktogramms aus d) sowie die Operationen aufstieg und sortiere (Abbildungen 7 und 10) wurden mit javac 21 und exakten Nachbauten der Niedersachsen-Klassen (Stack, Queue, DynArray, BinTree – Object-basiert, gleiche Exceptions) übersetzt und mit der Testklasse KletterhalleTest geprüft (29 Tests: c 7, d 5, e 5, f 5, m 7). Alle Zahlen der Klausur und des Erwartungshorizonts wurden zusätzlich mit einem unabhängigen Python-Modell berechnet und verglichen. Die folgende Tabelle zeigt, wie die Tests auf gleichwertige Alternativen (müssen grün sein) und typische Fehlvarianten (müssen mindestens einen roten Test erzeugen) reagieren.

Teil	Art	Variante	P	Tests grün	rote Tests
c)	Alternative	Zählschleife mit Leerabfrage in der Bedingung		7/7	
c)	Alternative	ganzen Stapel umschichten, nur die ersten k auswerten		7/7	
c)	Fehlvariante	keine Leerabfrage in der Schleifenbedingung	P1	4/7	kGroesserAlsAnzahl, leeresTourenbuch, unveraendertGanz
c)	Fehlvariante	Tourenbuch nicht wiederhergestellt	P2	3/7	beispiel, grenzeGenau, unveraendertGanz, unveraendertTeilweise
c)	Fehlvariante	Zurückschichten über eine Hilfsschlange (Reihenfolge umgekehrt)	P2	5/7	unveraendertGanz, unveraendertTeilweise
c)	Fehlvariante	tourenbuch = hilf statt Zurückschichten	P2	2/7	beispiel, grenzeGenau, kNull, unveraendertGanz, unveraendertTeilweise
c)	Fehlvariante	anzahl <= k (eine Route zu viel)	P6	4/7	beispiel, grenzeGenau, kNull
c)	Fehlvariante	Startwert top() ohne Leerabfrage	P1	5/7	kNull, leeresTourenbuch
c)	Fehlvariante	return beim Erreichen von k vor dem Zurückschichten	P2/P3	4/7	beispiel, grenzeGenau, unveraendertTeilweise
c)	Fehlvariante	Cast (Route) vergessen	P4	Kompilierfehler	(Kompilierfehler)
d)	Alternative	Rotieren mit vorher bestimmter Länge (ohne Hilfsschlange)		5/5	
d)	Fehlvariante	alle Mitglieder mit dem Namen entfernt (! gefunden fehlt)		3/5	beispiel, zweimal
d)	Fehlvariante	== statt equals	P5	2/5	beispiel, ersterUndLetzter, zweimal
d)	Fehlvariante	Hilfsschlange nicht zurückgeschichtet	P2	1/5	beispiel, ersterUndLetzter, nichtVorhanden, zweimal
d)	Fehlvariante	Zurückschichten über einen Stapel (Reihenfolge umgekehrt)	P2	2/5	beispiel, nichtVorhanden, zweimal
d)	Fehlvariante	Abbruch beim Treffer (Rest der Schlange geht verloren)	P2	3/5	beispiel, zweimal
e)	Alternative	Rückwärts laufende Zählschleife		5/5	
e)	Alternative	Zählschleife mit i-- nach delete		5/5	
e)	Fehlvariante	delete in Vorwärtsschleife, i wird immer erhöht	P6	3/5	alle, beispielDach
e)	Fehlvariante	Länge vor der Schleife festgelegt	P1/P6	2/5	alle, beispielDach, ersteUndEinzelne
e)	Fehlvariante	Schleifengrenze <= getLength()	P6	0/5	alle, beispielDach, ersteUndEinzelne, keinTreffer, leer
e)	Fehlvariante	== statt equals	P5	2/5	alle, beispielDach, ersteUndEinzelne

Teil	Art	Variante	P	Tests grün	rote Tests
e)	Fehlvariante	nur gezählt, nicht entfernt		2/5	alle, beispielDach, ersteUndEinzelne
f)	Alternative	Startwert per Suche nach erster Route im Sektor		5/5	
f)	Fehlvariante	Fehlerhafte Fassung aus Abbildung 5 (unverändert)	P1/P5/P6	0/5	beispielKante, dach, keinTreffer, leer, platteErsteRoute
f)	Fehlvariante	nur Schleifengrenze und equals korrigiert (Startwert bleibt)	P1	2/5	beispielKante, keinTreffer, leer
f)	Fehlvariante	Startwert null, aber Vergleich ohne null-Abfrage	P1	2/5	beispielKante, dach, platteErsteRoute
f)	Fehlvariante	Startwert null, Schleife ab i = 1	P6	4/5	platteErsteRoute
m)	Alternative	Kandidat als Parameter (Hilfsoperation)		7/7	
m)	Alternative	iterativ mit Kandidat (Vorgabe „rekursiv“ nicht erfüllt)		7/7	
m)	Teillösung	ganzen Baum durchsucht (Ordnung nicht genutzt)	AFB III	6/7	nutztSuchbaumOrdnung
m)	Fehlvariante	Basisfall isEmpty fehlt	P1/P7	0/7	beispiel, kandidatOben, keiner, leer, nutztSuchbaumOrdnung, strengGroesser, unveraendert
m)	Fehlvariante	Ergebnis des linken Teilbaums nicht verwendet	P7	4/7	beispiel, kandidatOben, strengGroesser
m)	Fehlvariante	Ergebnis links ohne null-Prüfung zurückgegeben	P7	4/7	beispiel, kandidatOben, strengGroesser
m)	Fehlvariante	< statt <= (gleiche Punktzahl als Ziel)	P6	2/7	beispiel, kandidatOben, keiner, nutztSuchbaumOrdnung, strengGroesser
m)	Fehlvariante	Richtung vertauscht		3/7	beispiel, kandidatOben, nutztSuchbaumOrdnung, strengGroesser
m)	Fehlvariante	Cast (Mitglied) vergessen	P4	Kompilierfehler	(Kompilierfehler)

Hinweis: Die Online-IDE vergleicht Zeichenketten auch mit == inhaltlich; dort bleiben die Tests bei == statt equals grün. In der Prüfung mit echtem Java werden sie rot – im Abitur ist equals verlangt (P5).

Papier-Compiler-Datei (nur Lehrkraft)

AQ_W6_Nachschreibklausur_Papier-Compiler.txt enthält die Kontextklassen Route, Mitglied, Station und Kletterhalle mit fünf Abtipp-Blöcken (c, d, e, f, m), die Pruefhilfe mit den Beispieldaten der Abbildungen, die Testklasse KletterhalleTest (Präfix = Teilaufgabe), ein Hauptprogramm mit den Kontrollwerten zu g) und i) und am Ende die Musterlösung in einem markierten Kommentarblock. Vor dem ersten Einsatz in der Online-IDE: Bibliothek „Niedersachsen“ aktivieren und den Auslieferungszustand einmal kompilieren (Tests von f) melden dann den Indexfehler der fehlerhaften Fassung).